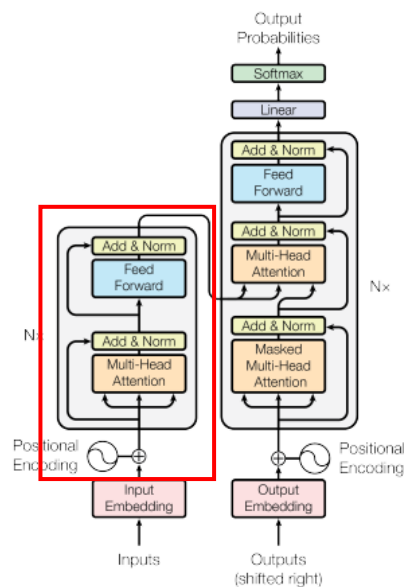


任務 1 : Transformer 模型

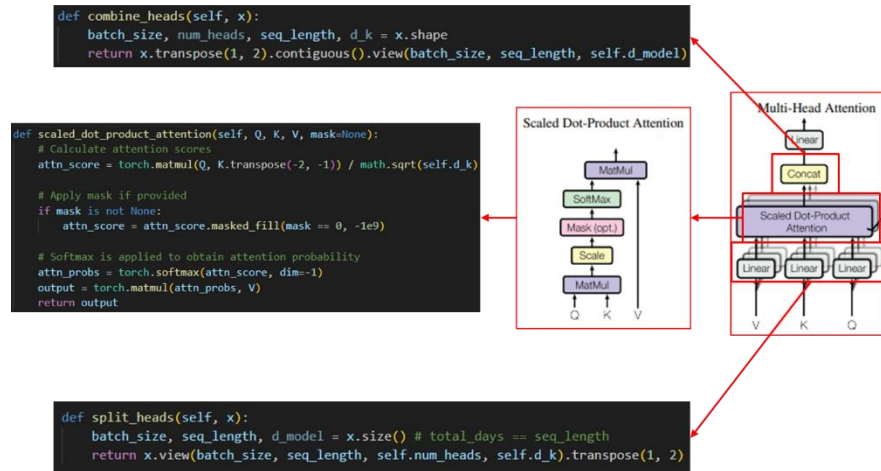
模型架構主要使用 Transformer 模型的 encoder block 來做特徵提取的步驟，最後面結合幾個 fully connect layer 來做價格的預測。以下為

Transformer 重要組件的介紹：



1. **PositionEncoding** : 負責給予數據位置資訊(0~29 天)
2. **Multi-Head Attention** : 將 Q, K, V(映射輸入而來)經過 split head 分成 h 組，之後個別輸入進各組對應的 Scale Dot-Product Attention 做運算，再來將 h 組做合併，通過全連接層映射和做 residual 跟

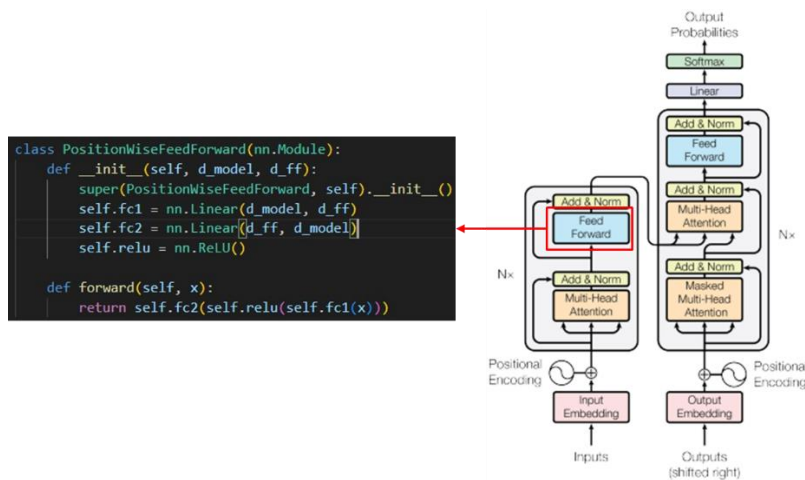
normalization 後再進入 FFN。



3. PositionWiseFeedForward(FFN) :

主要執行流程為：全連接層→激活層→全連接層

→residual→normalization

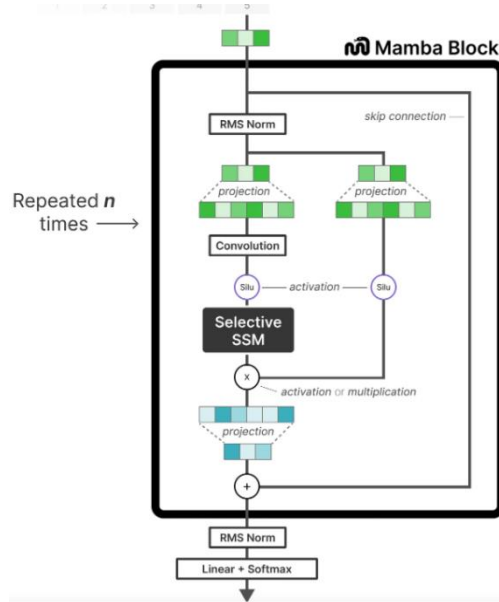


4. 額外設置：

- i. 最後的全連接層(留)：Encoder 的部分結束後，將張量為 [batch_size, 30 天, d_model] 的特徵壓縮成 [batch_size, 1, d_model]，之後進入 3 層全連接層生成輸出(輸出張量：[batch_size, 1, 4])生成最終輸出。

任務 2 : Mamba 模型(優化任務 1 的模型)

架構主要參考 Mamba 論文中的架構實作，將原 Transformer 中的 encoder block 換成 mamba block，以下為重要組件介紹：



1. Selective SSM(S6)：將原 SSM(Structured State Space Sequence

Model, S4)改造成其參數(A, B, C, delta)會隨著輸入的變化而影響，進

而達成模型的靈活性(模型能根據輸入內容選擇性將其記住或忽略)。

Algorithm 1 SSM (S4)

Input: $x : (B, L, D)$
Output: $y : (B, L, D)$
 1: $A : (D, N) \leftarrow \text{Parameter}$
 $\triangleright$ Represents structured $N \times N$ matrix
 2: $B : (D, N) \leftarrow \text{Parameter}$
 3: $C : (D, N) \leftarrow \text{Parameter}$
 4: $\Delta : (D) \leftarrow \tau_{\Delta}(\text{Parameter})$
 5: $\bar{A}, \bar{B} : (D, N) \leftarrow \text{discretize}(\Delta, A, B)$
 6: $y \leftarrow \text{SSM}(\bar{A}, \bar{B}, C)(x)$
 $\triangleright$ Time-invariant: recurrence or convolution
 7: **return** y

Algorithm 2 SSM + Selection (S6)

Input: $x : (B, L, D)$
Output: $y : (B, L, D)$
 1: $A : (D, N) \leftarrow \text{Parameter}$
 $\triangleright$ Represents structured $N \times N$ matrix
 2: $B : (B, L, N) \leftarrow s_B(x)$
 3: $C : (B, L, N) \leftarrow s_C(x)$
 4: $\Delta : (B, L, D) \leftarrow \tau_{\Delta}(\text{Parameter} + s_{\Delta}(x))$
 5: $\bar{A}, \bar{B} : (B, L, D, N) \leftarrow \text{discretize}(\Delta, A, B)$
 6: $y \leftarrow \text{SSM}(\bar{A}, \bar{B}, C)(x)$
 $s_{\Delta}(x) = \text{Broadcast}_D(\text{Linear}_1(x))$, and $\tau_{\Delta} = \text{softplus}$,
 7: **return** y

i. B, C, delta 參數：

```
# projects x to -dependent Δ, B, C
self.x_proj = nn.Linear(config.d_inner, config.dt_rank + 2 * config.d_state, bias=False)
```

$s_B(x) = \text{Linear}_N(x)$, $s_C(x) = \text{Linear}_N(x)$,

```
delta, B, C = torch.split(deltaABC,
                           [self.config.dt_rank, self.config.d_state, self.config.d_state], dim=-1)
delta = F.softplus(self.dt_proj(delta))
```

1: $A : (D, N) \leftarrow \text{Parameter}$
 $\triangleright$ Represents structured $N \times N$ matrix
 2: $B : (B, L, N) \leftarrow s_B(x)$
 3: $C : (B, L, N) \leftarrow s_C(x)$
 4: $\Delta : (B, L, D) \leftarrow \tau_{\Delta}(\text{Parameter} + s_{\Delta}(x))$

ii. 離散化(1a→2a)：將 A, B 離散化成 $\bar{A}$, $\bar{B}$ 。

$$\begin{aligned} h'(t) &= Ah(t) + Bx(t) & (1a) & \quad h_t = \bar{A}h_{t-1} + \bar{B}x_t & (2a) \\ y(t) &= Ch(t) & (1b) & \quad y_t = Ch_t & (2b) \end{aligned} \quad \bar{A} = \exp(\Delta A) \quad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$$

```
deltaA = torch.exp(delta.unsqueeze(-1) * A)
deltaB = delta.unsqueeze(-1) * B.unsqueeze(2)
```

iii. 序列掃描(S6 運算)：主要時做的部分為(方程式 2a, 2b 的部分) $h_t = \bar{A}h_{t-1} + \bar{B}x_t$ (2a)
 $y_t = Ch_t$ (2b)

```
def selective_scan_seq(self, x, delta, A, B, C):
    _, L, _ = x.shape
    Bx = B * (x.unsqueeze(-1)) # (B, L, N, N)

    h = torch.zeros(x.size(0), self.d_model, self.d_model, device=A.device)

    hs = []
    for t in range(0, L):
        h = A[:, t] * h + Bx[:, t]
        hs.append(h)
    hs = torch.stack(hs, dim=1) # (B, L, N, N)

    y = (hs @ C.unsqueeze(-1)).squeeze(3) # (B, L, N, N) @ (B, L, N, 1) -> (B, L, N)
    return y
```

2. 其他組件：Convolution Layer、RMS Normalization Layer、Silu
 activation、Projction(Fully Connected Layer)

3. Tuning Mamba(一開始效果非常差)：

```
[最高, 最低, 開盤, 收盤]
result: tensor([[11320.7773, -7889.2363, 5672.5889, 836.5897]]),
grad_fn=<SqueezeBackward1>
target: tensor([[777., 753., 771., 753.]])
val loss: 7243.82568359375
推論時間: 0.0390017032623291 seconds
Total Parameters: 27410
Trainable Parameters: 27410
```

i. 輸入模型前先用 normalization：為了縮小資料的數值，期望模

型能加速收斂(推論時間降低，RMS 也降低)。

```
[最高, 最低, 開盤, 收盤]
result: tensor([[890.4307, 744.5018, 837.3010, 886.2328]]),
target: tensor([[777., 753., 771., 753.]])
val loss: 93.65568542480469
推論時間: 0.03769397735595703 seconds
Total Parameters: 27410
Trainable Parameters: 27410
```

ii. 額外懲罰(延續 i)：加入一些期望模型能預測的方向，如未達成

loss 會增加。(例：價格不能小於 0、最高價不能低於其他價格、

最低價不能高於其他價格...等)。效果反而更差

```
[最高, 最低, 開盤, 收盤]
result: tensor([[[123.0863, 104.4584, 71.4742, 128.2468]]], grad_fn=<SqueezeBackward1>)
target: tensor([[[777., 753., 771., 753.]]])
val loss: 657.2408447265625
推論時間: 0.03602743148803711 seconds
Total Parameters: 27410
Trainable Parameters: 27410
```

任務 3(推論數據 輸入：20240316 前 30 天數據，輸出：20240316 當天數據)：

比較類別\模型	Transformer(左圖)	Mamba(優化 i)
大小	0.93M	0.027M
RMSE	303.418	93.66
推論時間(s)	0.055	0.038

```
[最高, 最低, 開盤, 收盤]
result: tensor([[[[306.7377, 353.0595, 189.8339, 325.9762]]]])
target: tensor([[[593., 589., 589., 593.]]])
val loss: 303.4177551269531
推論時間: 0.055590152740478516 seconds
Total Parameters: 927052
Trainable Parameters: 927052
```

```
[最高, 最低, 開盤, 收盤]
result: tensor([[[890.4307, 744.5018, 837.3010, 886.2328]]],
target: tensor([[[777., 753., 771., 753.]]])
val loss: 93.65568542480469
推論時間: 0.03769397735595703 seconds
Total Parameters: 27410
Trainable Parameters: 27410
```

執行方法

1. 打開終端機進入 model 資料夾中
2. 輸入：bash start.sh
3. 開始執行